

```

//
// Name: Keyanna Takita
// CS 372
// main()

#include <iostream>
#include <utility>
#include "List.hpp"
#include "Bag.hpp"
#include "CircularList.hpp"
using namespace std;

// ----- Default Constructor -----
/* int main()
{
    List<int> numbers;

    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_front(5);

    cout << "Front: " << numbers.front() << endl;
    cout << "Back: " << numbers.back() << endl;

    numbers.pop_front();
    cout << "Front after pop_front: " << numbers.front() << endl;

    numbers.pop_back();
    cout << "Back after pop_back: " << numbers.back() << endl;

    return 0;
}
*/

// ----- Destructor -----
/* int main()
{
    {
        List<int> numbers;

        numbers.push_back(10);
        numbers.push_back(20);
        numbers.push_back(30);
    }

    cout << "List destroyed successfully" << endl;

    return 0;
}
*/

```

```
// ----- Copy Constructor -----
/* int main()
{
    List<int> numbers;
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    List<int> copyNumbers = numbers;

    cout << "Original front: " << numbers.front() << endl;
    cout << "Copy front: " << copyNumbers.front() << endl;

    numbers.pop_front();

    cout << "Original front after pop: " << numbers.front() << endl;
    cout << "Copy front after original pop: " << copyNumbers.front() << endl;

    return 0;
};
*/
```

```
// ----- Copy Assignment -----
/*int main()
{
    List<int> listA;
    listA.push_back(1);
    listA.push_back(2);

    List<int> listB;
    listB.push_back(100);

    listB = listA;

    cout << "List A front: " << listA.front() << endl;
    cout << "List B front: " << listB.front() << endl;

    listA.pop_front();

    cout << "List A after pop: " << listA.front() << endl;
    cout << "List B after A pop: " << listB.front() << endl;
    return 0;
}
*/
```

```
// ----- Move Constructor -----

/* int main()
{
```

```

    List<int> listA;
    listA.push_back(50);
    listA.push_back(60);

    List<int> listB = move(listA);

    cout << "List B front after move: " << listB.front() << endl;
    cout << "List B back after move: " << listB.back() << endl;

    return 0;
}
*/

// ----- Move Assignement -----
/* int main()
{

    List<int> listA;
    listA.push_back(100);
    listA.push_back(200);

    List<int> listB;
    listB.push_back(1);

    listB = move(listA);

    cout << "Front: " << listB.front() << endl;
    cout << "Back: " << listB.back() << endl;

    return 0;
}
*/

// ----- Add Traverse ----
/* void printNumber(int number)
{
    cout << number << endl;
}

int main()
{
    List<int> numbers;

    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    numbers.traverse(printNumber);

    return 0;
}

```

```

*/

// ----- Bag -----
/*
void printItem(int item)
{
    cout << item << endl;
}

int main()
{
    Bag<int> myBag;

    myBag.add(10);
    myBag.add(20);
    myBag.add(30);

    cout << "Bag contents:" << endl;
    myBag.traverse(printItem);

    cout << "Grabbed: " << myBag.grab() << endl;

    cout << "Bag after grab:" << endl;
    myBag.traverse(printItem);

    return 0;
}
*/

// ----- Circular List (Loop) -----
void printItem(int item)
{
    cout << item << endl;
}

int main()
{
    CircularList<int> numbers;

    numbers.push_back(1);
    numbers.push_back(2);
    numbers.push_back(3);

    numbers.traverseFromFront(printItem, 2);

    return 0;
}

```